

APTITUDEMIND

Adaptive Agentic AI Mentor

Complete Development Documentation

Work Completed So Far

MILESTONE 2: PERSISTENT MEMORY

This document preserves the complete original AptitudeMind project vision. Milestones reflect implementation order only and do not represent a reduction in project scope.

1. Project Identity

Project Name: AptitudeMind

Full Name: AptitudeMind — An Adaptive Agentic AI Mentor

AptitudeMind is being developed as an intelligent aptitude-learning mentor.

The project is not being built by simply copying an existing complete agent implementation.

Development Philosophy

Learn the concept
→ Understand the architecture
→ Write the code
→ Test
→ Debug
→ Improve

The objective is to understand how an agent actually works internally.

2. Original Project Vision — Must Be Preserved

The original AptitudeMind vision **MUST NOT** be reduced or removed because of deadlines or implementation difficulty.

The complete original roadmap is:

- LLM fundamentals
- Tools / function calling
- Prompt engineering
- Agent workflow & decision making
- Memory
- RAG / knowledge retrieval
- Multi-step / autonomous tasks
- Error handling & guardrails
- Testing & evaluation
- API + deployment
- Clean UI
- Documentation
- GitHub
- Final presentation

NONE OF THESE FEATURES ARE TO BE REMOVED.

Intended Behavior of AptitudeMind

Student asks/answers an aptitude question
↓
AptitudeMind understands the student's situation

↓
Checks level / weak topic / recent performance
↓
Chooses or generates an appropriate question
↓
Student answers
↓
AptitudeMind evaluates the answer
↓
Records performance
↓
Learns from the student's performance
↓
Adapts future questions

The long-term objective is to create a personalized adaptive aptitude mentor.

3. Development Environment

- **Operating System:** Ubuntu
- **Python:** Python 3.12.3
- **IDE:** Visual Studio Code
- **Project virtual environment:** .venv

The project uses a local-first AI approach because paid API access is not being relied upon.

The local LLM framework selected is: **Ollama**

4. Ollama Setup

Ollama was successfully installed. Ollama version used: **0.33.3**

The local model used: **Llama 3.2 3B**

The model was downloaded using:

```
ollama pull llama3.2:3b
```

The model was successfully run locally using:

```
ollama run llama3.2:3b
```

The Python Ollama library was also installed.

This established the local AI foundation for AptitudeMind.

5. Initial Project Structure

The project was initially created as:

```
AptitudeMind
■■■ .venv
■■■ .env
■■■ .gitignore
■■■ main.py
```

■■■ README.md

The .gitignore included protection for:

```
.venv/  
.env  
__pycache__/  
*.pyc
```

6. First LLM Integration

The first version of main.py connected Python to the local Llama model using:

```
ollama.chat()
```

The model: **llama3.2:3b**

The first objective was simple: *Ask the LLM to generate an aptitude question.*

This was the first step toward turning AptitudeMind into an intelligent aptitude mentor.

7. Prompt Engineering Experiments

Early testing revealed that LLM-generated aptitude questions could be:

- Ambiguous
- Incomplete
- Semantically incorrect
- Difficult to calculate reliably
- Different from what the program expected

This was an important engineering discovery. The prompt was therefore progressively strengthened.

The improved prompt required:

- One question only
- Easy-level aptitude question
- Self-contained question
- One clear numerical answer
- Explicit numerical values
- No ambiguity
- No unnecessary explanation
- Strict output format
- Meaningful use of the provided numbers

Good and bad examples were also added to the prompt.

This demonstrated that prompt engineering can improve LLM output quality.

However, an important lesson was learned: Prompt engineering alone is NOT enough to guarantee mathematical correctness.

8. Structured LLM Output

The project moved from free-form LLM responses toward structured output. Initially the structure was:

```
QUESTION:
VALUE:
PERCENTAGE:
```

The purpose was to allow Python to extract the important information programmatically.

Instead of blindly trusting an LLM-generated numerical answer, Python would perform the actual mathematical calculation. This created the architecture:

```
LLM
↓
Generate question
↓
Extract numbers
↓
Python calculator
↓
Verify correct answer
↓
Store validated Q&A
↓
Student answers
↓
Evaluate reliably
```

9. Calculator Tool

A separate file was created: **calculator.py**

The first function was:

```
calculate_percentage(value, percentage)
```

The mathematical operation was: $\text{VALUE} \times \text{PERCENTAGE} / 100$

Example: 25% of 200 = $200 \times 25 / 100 = 50$

The purpose was to separate deterministic mathematical computation from probabilistic LLM generation.

This was an important engineering principle:

```
LLM    → generation/reasoning assistance
Python → deterministic calculation
```

10. First Major Engineering Bug

A major bug was discovered during testing.

The LLM generated a discount question such as:

```
"A shirt is on sale for 30% off its original price of $120. How much will you pay for the shirt?"
```

The student correctly answered: **84**

But the calculator initially calculated: $30\% \text{ of } 120 = 36$

Therefore the application incorrectly said the answer was 36.

This was NOT a mathematical mistake by the student. The student's answer 84 was correct.

The actual issue was: **SEMANTIC MISMATCH**

The calculator calculated: Discount amount = 36

But the question asked for: Final price = 84

This was a critical engineering lesson.

11. Lesson From the Semantic Mismatch

Simply extracting numbers from an LLM-generated question is not enough.

The system must understand what operation the question is asking for. For example:

"What is 30% of 120?"
requires: $120 \times 30 / 100 = 36$

"A \$120 shirt is 30% off. What is the final price?"
requires: $120 - (120 \times 30 / 100) = 84$

Therefore the agent needs semantic information about the type of operation.

12. Operation Type Was Introduced

The structured LLM output was upgraded. The format became:

QUESTION:
TYPE:
VALUE:
PERCENTAGE:

TYPE must be one of:

- PERCENTAGE
- DISCOUNT
- INCREASE

13. Percentage Operation

For TYPE: PERCENTAGE, the system calculates:

$\text{VALUE} \times \text{PERCENTAGE} / 100$

Example:

QUESTION: What is 25% of \$200?
TYPE: PERCENTAGE
VALUE: 200
PERCENTAGE: 25

Answer: 50

14. Discount Operation

For TYPE: DISCOUNT, the system calculates the final price after discount.

Formula:

$$\text{VALUE} - (\text{VALUE} \times \text{PERCENTAGE} / 100)$$

Example:

QUESTION: A shirt costs \$200 and is available at 25% off. What is the final price?

TYPE: DISCOUNT

VALUE: 200

PERCENTAGE: 25

Calculation:

Discount = $200 \times 25 / 100 = 50$

Final price: $200 - 50 = 150$

15. Increase Operation

For TYPE: INCREASE, the system calculates the final value after percentage increase.

Formula:

$$\text{VALUE} + (\text{VALUE} \times \text{PERCENTAGE} / 100)$$

Example:

QUESTION: A salary is \$200 and increases by 25%. What is the new salary?

TYPE: INCREASE

VALUE: 200

PERCENTAGE: 25

Calculation:

Increase = $200 \times 25 / 100 = 50$

New value: $200 + 50 = 250$

16. Current calculator.py

The calculator.py module was upgraded to contain:

- `calculate_percentage(value, percentage)`
- `calculate_discount(value, percentage)`
- `calculate_increase(value, percentage)`

The functions are:

- **`calculate_percentage`**: Returns the percentage amount.
- **`calculate_discount`**: Calculates the final value after discount.
- **`calculate_increase`**: Calculates the final value after increase.

This creates a clean separation between LLM question generation and deterministic mathematical calculation.

17. Current main.py Behavior

The current main.py:

- Starts AptitudeMind.
- Sends a strict prompt to Llama 3.2 3B.
- Requests one aptitude question.
- Requests structured output.
- Extracts QUESTION, TYPE, VALUE, PERCENTAGE.
- Displays only the question to the student.
- Keeps internal calculation information hidden.
- Determines the correct calculator operation from TYPE.
- Calculates the correct answer using Python.
- Accepts the student's numerical answer.
- Compares the student's answer with the verified Python answer.
- Uses a small tolerance for floating-point differences.
- Displays correct/incorrect feedback.
- Provides a step-by-step explanation when the student is wrong.
- Records the result.
- Displays progress.

18. Student-Facing Question

The student should only see the actual question. For example:

■ Question:

A shirt costs \$200 and is on sale at 15% off. What is the final price?

The internal values TYPE, VALUE, PERCENTAGE, and CORRECT ANSWER are used internally by the program and are not shown to the student before answering.

19. Numerical Answer Evaluation

The program converts the student's input into a floating-point value. A tolerance is used:

```
abs(student_value - correct_answer) < 0.01
```

This prevents tiny floating-point representation differences from causing an otherwise correct answer to be marked wrong.

20. Progress Tracking — First Version

A file named **progress.py** was created.

The first progress system stored:

- total
- correct
- wrong

The functions included:

- record_result(is_correct)
- show_progress()

The progress calculation was:

```
accuracy = correct / total × 100
```

Testing successfully produced:

```
Questions attempted: 3
Correct answers: 2
Wrong answers: 1
Accuracy: 66.67 %
```

This proved that the basic progress tracking system worked.

21. Limitation of First Progress System

The first progress implementation stored data only in memory. Therefore:

```
Program running: Progress exists.
Program closed: Progress disappears.
```

This was identified as insufficient for a real adaptive mentor. A real mentor must remember the student's previous performance.

22. Persistent Memory — Milestone 2

The progress system was upgraded to persistent memory.

A new file was created: **memory.py**

A persistent data file is: **progress.json**

The new architecture is:

```
main.py
↓
progress.py
↓
memory.py
↓
progress.json
```

23. memory.py Responsibilities

memory.py provides persistent memory functionality. It can:

- Load previous progress
- Create default memory if no progress file exists
- Record correct answers
- Record wrong answers
- Update total attempts
- Save progress to progress.json
- Display progress
- Continue previous progress after restarting the program

The memory currently contains: total, correct, wrong

24. Current Persistent Memory Structure

Example progress.json:

```
{
  "total": 1,
  "correct": 1,
  "wrong": 0
}
```

If another question is answered correctly:

```
{
  "total": 2,
  "correct": 2,
  "wrong": 0
}
```

The important improvement is that this information survives program restarts.

25. progress.py After Memory Upgrade

progress.py now acts as a simple interface between the main application and memory.py.

It imports **record_result** and **show_progress** from **memory.py**.

This creates a clean separation of responsibilities.

26. Personal Data Protection

progress.json contains personal learning progress. Therefore it should NOT be uploaded to the public GitHub repository.

The .gitignore was updated to include: progress.json

The final .gitignore protects:

```
.venv/  
.env  
__pycache__/  
*.pyc  
progress.json
```

This means:

- Source code → GitHub
- Personal progress → local computer

This is an important professional practice.

27. Current Project Structure

```
AptitudeMind/  
■  
■■■■ .venv/  
■  
■■■■ .env  
■  
■■■■ main.py  
■  
■■■■ calculator.py  
■  
■■■■ progress.py  
■  
■■■■ memory.py  
■  
■■■■ progress.json  
■  
■■■■ .gitignore  
■  
■■■■ README.md
```

Important: progress.json is local and ignored by Git.

28. Verified Test — Discount Question

AptitudeMind successfully generated:

"A shirt costs \$200 and is on sale at 15% off. What is the final price?"

The student entered: **170**

The correct calculation is:

```
15% of 200 = 200 × 15 / 100 = 30  
Final price: 200 - 30 = 170
```

AptitudeMind correctly responded:

```
■ Correct! Excellent work! ■  
■ You calculated the answer correctly.
```

Progress displayed:

Questions attempted: 1
Correct answers: 1
Wrong answers: 0
Accuracy: 100.0 %

This verified that the semantic mismatch problem had been successfully fixed for the supported discount operation.

29. Git Initialization

Git was initialized for the AptitudeMind project.

The Git identity was configured as:

- **Name:** Karthik R P
- **Email:** karthukarthik242@gmail.com

The main branch was created.

The remote GitHub repository was configured as:

```
https://github.com/KarthikRP424/AptitudeMind.git
```

30. Initial GitHub Push

The initial AptitudeMind implementation was committed.

Commit message:

```
Initial AptitudeMind agent implementation
```

The main branch was pushed successfully to GitHub.

31. Git Push Issue and Safe Solution

Later, after making new changes, a push initially failed with:

```
! [rejected] main -> main (fetch first)
```

Git explained that the remote contained work that was not present locally.

Instead of force pushing, the safe approach was used:

```
git pull --rebase origin main
```

After integrating the remote history, the project was pushed again.

This is important because force pushing could overwrite remote work.

32. Final Successful GitHub Push

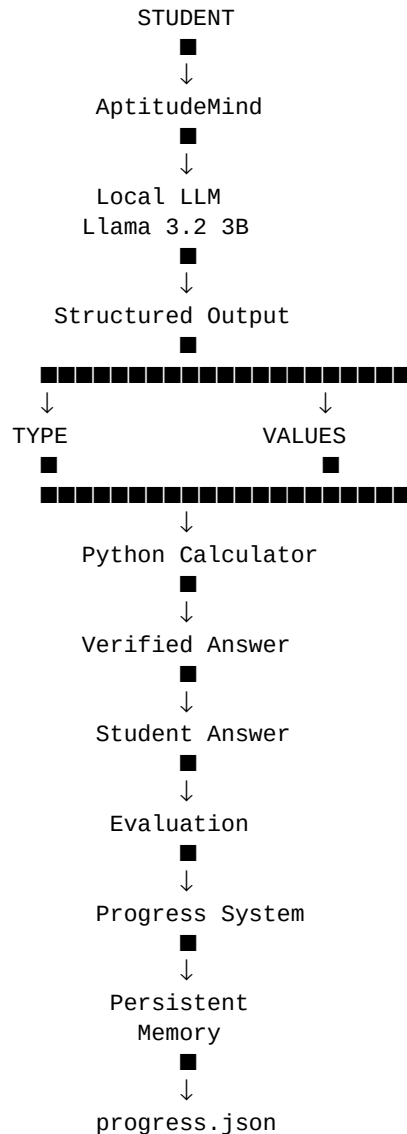
The final push succeeded. Output included:

```
Enumerating objects: 11, done.  
Counting objects: 100% (11/11), done.
```

```
Delta compression using up to 8 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (7/7), 3.09 KiB | 791.00 KiB/s, done.
Total 7 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/KarthikRP424/AptitudeMind.git
b63aed8..ddb84b9  main -> main
```

This confirms that the current AptitudeMind implementation was successfully pushed to GitHub.

33. Current Architecture



34. Responsibility Separation

The project now separates different responsibilities.

- **LLM:** Question generation.

- **Prompt:** Controls expected question structure and behavior.
- **Structured output:** Provides QUESTION, TYPE, VALUE and PERCENTAGE.
- **Python calculator:** Performs deterministic mathematical calculations.
- **Evaluation engine:** Checks the student's answer.
- **Progress system:** Records performance.
- **Memory:** Persists performance between sessions.

This is a strong foundation for a real agent architecture.

35. What Has Been Proven

- Local LLM integration.
- LLM-based aptitude question generation.
- Prompt engineering.
- Structured LLM output.
- Deterministic Python calculations.
- Percentage calculation.
- Discount calculation.
- Increase calculation.
- Semantic operation selection.
- Student answer evaluation.
- Numerical tolerance handling.
- Correct/incorrect feedback.
- Step-by-step explanations.
- Progress tracking.
- Persistent memory.
- Git version control.
- GitHub integration.
- Successful GitHub push.

36. Important Engineering Lessons

LESSON 1: LLMs are powerful generators, but their outputs must be validated.

LESSON 2: Prompt engineering improves output quality but cannot replace deterministic verification.

LESSON 3: Structured outputs make LLM integrations easier to process programmatically.

LESSON 4: Mathematical operations that require guaranteed correctness should be handled by deterministic code.

LESSON 5: Semantic alignment between the question and the tool operation is essential.

LESSON 6: Simply extracting numbers is not enough.

LESSON 7: An agent must understand what operation is actually being requested.

LESSON 8: Memory is a core component of an adaptive agent.

LESSON 9: Personal runtime data should be separated from public source code.

LESSON 10: Git provides a safe history of engineering changes.

LESSON 11: When GitHub rejects a push because remote history differs, pulling with rebase is safer than blindly force pushing.

37. What We Are Not Removing

The following future capabilities **MUST** remain part of AptitudeMind:

- LLM fundamentals
- Tools / function calling
- Prompt engineering
- Agent workflow
- Decision making
- Memory
- RAG
- Knowledge retrieval
- Multi-step tasks
- Autonomous tasks
- Error handling
- Guardrails
- Testing
- Evaluation
- API
- Deployment
- Clean UI
- Documentation
- GitHub
- Final presentation

The project scope remains unchanged.

38. Current Milestone Status

Milestone 1

- LLM integration

- Prompt engineering
- Structured output
- Calculator tool
- Percentage operation
- Discount operation
- Increase operation
- Reliable operation-aware evaluation

STATUS: COMPLETED

Milestone 2

- Persistent student memory

STATUS: COMPLETED

39. Next Development Milestone

The next milestone is: **TOPIC MEMORY + WEAK TOPIC DETECTION**

Currently AptitudeMind remembers only: Total questions, Correct answers, Wrong answers, Overall accuracy.

The next upgrade should allow AptitudeMind to remember topic-level performance. Example:

```
Percentage
  Attempted: 5
  Correct: 4
  Accuracy: 80%

Profit & Loss
  Attempted: 4
  Correct: 2
  Accuracy: 50%

Time & Work
  Attempted: 3
  Correct: 3
  Accuracy: 100%
```

Then AptitudeMind should be able to identify:

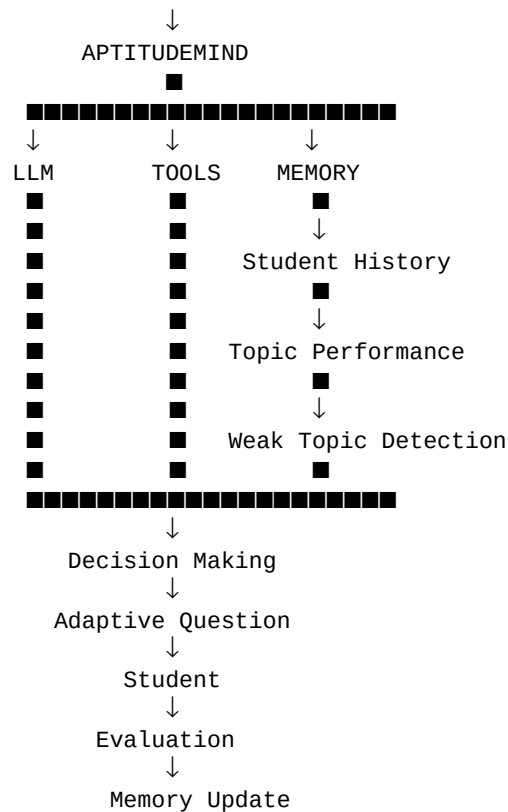
- **Weak topic:** Profit & Loss
- **Strong topic:** Time & Work

And eventually use that information to generate more suitable questions.

40. Future Adaptive Agent Architecture

The planned future architecture is:

STUDENT
■



41. Long-Term AptitudeMind Vision

The final objective is not merely: “An LLM that generates aptitude questions.”

The objective is:

“An adaptive agentic AI mentor that understands a student's performance, remembers their learning history, identifies weaknesses, chooses appropriate learning tasks, evaluates answers reliably, and continuously adapts.”

The long-term system should eventually include:

- LLM reasoning
- Tool use
- Function calling
- Memory
- RAG
- Knowledge retrieval
- Topic tracking
- Weak-topic detection
- Difficulty adaptation
- Multi-step workflows
- Autonomous decision making

- Error handling
- Guardrails
- Evaluation
- API
- Deployment
- Clean user interface

42. Development Principle

AptitudeMind is being developed step-by-step. Every milestone must be:

```
UNDERSTOOD
→ IMPLEMENTED
→ TESTED
→ DEBUGGED
→ IMPROVED
→ DOCUMENTED
```

Milestones are implementation stages. They are NOT reductions in the original project scope.

The complete original AptitudeMind vision remains the final target.

43. Final Summary of Work Completed So Far

AptitudeMind has progressed from a basic idea into a working local AI application.

```
Starting point:    LLM question generation
Then:             Prompt engineering
Then:             Structured output
Then:             Python calculator
Then:             Semantic operation handling
Then:             Reliable student answer evaluation
Then:             Progress tracking
Then:             Persistent memory
Then:             Git version control
Then:             GitHub integration
```

The current system can generate a percentage-related question using a local LLM, determine the mathematical operation, calculate the correct answer using Python, evaluate the student's response, record performance, and persist basic progress between sessions.

This is the foundation on which the larger adaptive agent will be built.

44. Final Status

AptitudeMind Status

[DONE]	Local LLM	WORKING
[DONE]	Ollama	WORKING
[DONE]	Llama 3.2 3B	WORKING
[DONE]	Prompt Engineering	COMPLETED

[DONE]	Structured Output	WORKING
[DONE]	Calculator Tool	WORKING
[DONE]	Percentage	WORKING
[DONE]	Discount	WORKING
[DONE]	Increase	WORKING
[DONE]	Answer Evaluation	WORKING
[DONE]	Progress Tracking	WORKING
[DONE]	Persistent Memory	WORKING
[DONE]	Git	WORKING
[DONE]	GitHub	WORKING
[DONE]	Successful Push	COMPLETED

Next

[NEXT]	Topic Memory
[NEXT]	Weak Topic Detection
[NEXT]	Adaptive Difficulty
[NEXT]	RAG
[NEXT]	Agent Workflow
[NEXT]	Multi-step / Autonomous Tasks
[NEXT]	Guardrails
[NEXT]	Testing & Evaluation
[NEXT]	API
[NEXT]	Deployment
[NEXT]	Clean UI
[NEXT]	Final Documentation
[NEXT]	Final Presentation

IMPORTANT FINAL NOTE

This documentation preserves the full original AptitudeMind vision. No roadmap item — including RAG, memory, autonomous tasks, tools, guardrails, testing, API/deployment, UI, documentation, GitHub, or the final presentation — has been removed from project scope. The purpose of this document is to maintain a permanent professional record of everything completed so far while preserving the full original AptitudeMind vision.